Subject Name: DESIGN AND ANALYSIS OF ALGORITHMS

Subject Code: 23CSE402

Prepared By: Ms. Preeti Gupta

Department: COMPUTER SCIENCE & ENGINEERING

Date:

OBJECTIVE

- This course aims in introducing recurrence relations, and illustrates their role in asymptotic and probabilistic analysis of algorithms. It covers in detail greedy strategies, divide and conquer techniques, dynamic programming and max flow - min cut theory for designing algorithms, and illustrates them using a number of well-known problems and applications.
- It also covers popular graph and matching algorithms, and basics of randomized algorithms and computational complexity.
- Basic knowledge of computational complexity, approximation and randomized algorithms.
- Teaches how to design efficient algorithms which leads to efficient programs.

COARSE OUTCOME

	Upon completion of this course, students will be able to do the following:
CO 1	Discuss the Characteristics of algorithm, the the notation of algorithms
CO 2	Illustrate the sorting, searching, graph traversal problem and divide and conquer design techniques.
CO 3	Solve the given real time problems through an greedy appropriate design strategy.
CO 4	Solve the combinatorial optimization problems which efficiently estimates the search space's lower and upper limits of branches..
CO 5	Illustrate the other design paradigms and apply.
CO6	Analyse the NP Complete problems.

Text Books:

1. Thomas H. Coreman, Charles E. Leiserson and Ronald L. Rivest, “Introduction to Algorithms”, Printice Hall ofIndia.
2. E. Horowitz & S Sahni, "Fundamentals of Computer Algorithms",
3. Aho, Hopcraft, Ullman, “The Design and Analysis of Computer Algorithms” Pearson Education, 2008.
4. LEE "Design & Analysis of Algorithms (POD)",McGraw Hill
5. Richard E.Neapolitan "Foundations of Algorithms" Jones & Bartlett Learning
6. Jon Kleinberg and Éva Tardos, Algorithm Design, Pearson, 2005.
7. Michael T Goodrich and Roberto Tamassia, Algorithm Design: Foundations, Analysis, and Internet Examples,Second Edition, Wiley, 2006.
8. Harry R. Lewis and Larry Denenberg, Data Structures and Their Algorithms, Harper Collins, 1997

Text Books

9. Robert Sedgewick and Kevin Wayne, Algorithms, fourth edition, Addison Wesley, 2011.
10. Harsh Bhasin,”Algorithm Design and Analysis”,First Edition,Oxford University Press.
11. Gilles Brassard and Paul Bratley,Algorithmics:Theory and Practice,Prentice Hall,1995.

TLEP

- Course Opening Report
- TLEP
- COLAB
- Syllabus
- Assignment1
- Assignment2
- IA-1 Marks
- Assignment3
- Assignment4
- IA-2 Marks
- Assignment5
- CA Marks
- Result
- Attainment
- Course Closing Report

Unit -1
INTRODUCTION

Unit-1 TOPICS

1. Why study Algorithm?
2. Algorithms
3. Analyzing Algorithms
4. Complexity of Algorithms
5. Growth of Functions
6. Performance Measurements
7. Sorting and Order Statistics-Shell Sort, Quick Sort, Merge Sort, Heap Sort, Comparison of Sorting Algorithms, Sorting in linear Time

ALGORITHM

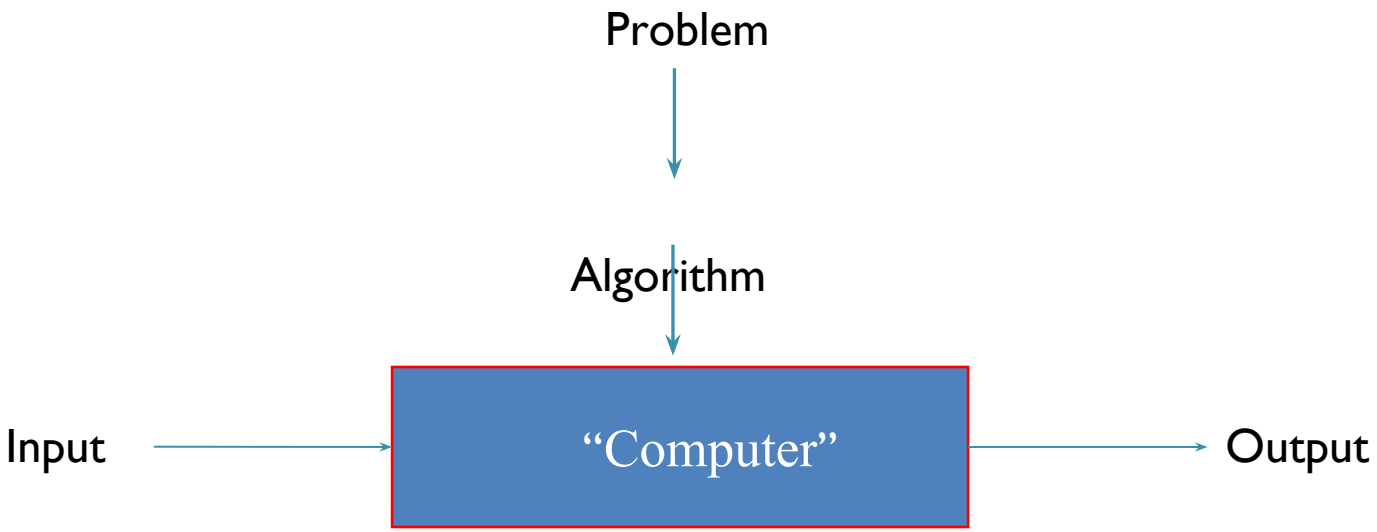
1. An **algorithm** is any well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**.
2. An algorithm is thus a sequence of computational steps that transform the input into the output.

An Algorithm should have some characteristics:

1. Input: It must have one or more input.
2. Output: It must produce some output.
3. Finiteness: Algorithm must complete after a finite number of instructions have been executed.
4. Efficient: Run as quickly as possible, and use as little memory as possible.
5. Ambiguity: Each step of algorithm must be clearly defined.

WHAT IS AN ALGORITHM?

An <u>algorithm</u> is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.



Design and Analysis of Algorithms

- ***Design:*** Design algorithms which minimize the cost
- ***Analysis:*** Predict the cost of an algorithm in terms of resources and performance

DESIGNING OF ALGORITHM

There are various method to design a algorithm:

1. Divide and Conquer Algorithm
2. Dynamic Programming
3. Greedy Algorithm
4. Branch and Bound Method
5. Backtracking Approach

METHODS FOR DESIGNING OF ALGORITHM

1. Divide and Conquer Algorithm: A problem is divided into number of sub-problems, solve each sub-problem recursively, combine the solution of each sub-problem to find the solution of original problem. It does more work on sub problems and hence has more time consumption. Example: Quick Sort, Merge Sort and Binary Search
2. Dynamic Programming: It is similar to divide and conquer algorithm. It solves subproblems only once and then stores in the table. For example: Matrix Multiplication.
3. Greedy Algorithm: A greedy algorithm is an approach for solving a problem by selecting the best option available at the moment. It works in a top-down approach. Example: Fractional Knapsack Problem, Kruskal's algorithm and Prim's algorithm for finding minimum spanning trees. Greedy algorithms appear in the network routing as well.

METHODS FOR DESIGNING OF ALGORITHM

4. Branch and Bound Method: It is one of the techniques used for problem solving. It uses state space tree and similar to the backtracking. It is used for solving the optimization problems and minimization problems. If we have given a maximization problem then we can convert it using the Branch and bound technique by simply converting the problem into a maximization problem. Example: Knapsack Problem, Assignment Problem
5. Backtracking Approach: Backtracking is a method of trying out various sequences of decision, until we find the one that works. For example: N-queens problem, Huffman Circuit problem, sub-set sum problem.

ANALYSIS OF ALGORITHMS

1. How good is the algorithm?
   - Correctness
   - Time efficiency
   - Space efficiency
2. Does there exist a better algorithm?
   - Lower bounds
   - Optimality

COMPLEXITY OF ALGORITHM

Complexity of an algorithm is a measure of the amount of time and/or space required by an algorithm for an input of a given size (n) to execute all steps of algorithm.

The complexity of an algorithm can be divided into two types. The time complexity and the space complexity.

1. Time Complexity of an Algorithm: The time complexity is defined as the process of determining a formula for total time required towards the execution of that algorithm. The unit of time complexity is Clock Cycle.
2. Space Complexity of an Algorithm: Space complexity is defined as how much memory space is required for the successful execution of the algorithm. The memory space is generally considered as the primary memory.

Complexity is calculated in three cases:

1) Best case 2)Worst case 3) Average case

Methodology of Analysis of Algorithm

1. **Growth of Functions and Asymptotic Notation**
   - When we study algorithms, we are interested in characterizing them according to their efficiency.
   - We are usually interesting in the order of growth of the running time of an algorithm, not in the exact running time. This is also referred to as the *asymptotic running time*.
   - We need to develop a way to talk about rate of growth of functions so that we can compare algorithms.
   - *Asymptotic notation* gives us a method for classifying functions according to their rate of growth.

Compare functions in the limit, that is, **asymptotically!**

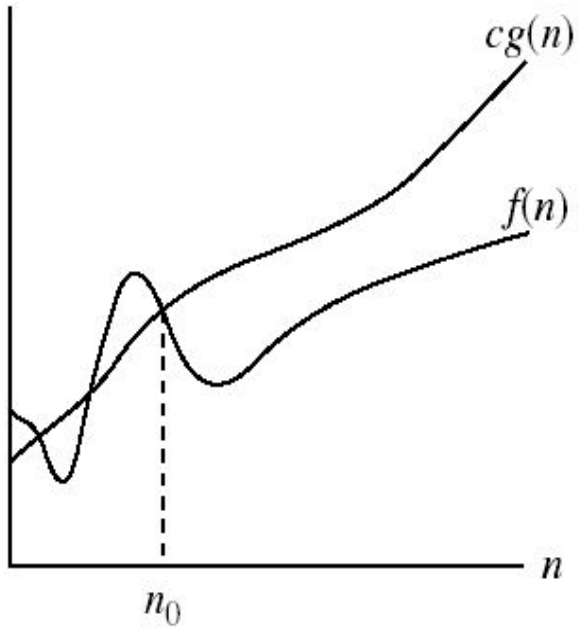
ASYMPTOTIC NOTATION

- **O** notation: asymptotic "less than":
  - $f(n)=O(g(n))$ implies: $f(n)$ "$\leq$" $g(n)$
- **Ω** notation: asymptotic "greater than":
  - $f(n)= \Omega (g(n))$ implies: $f(n)$ "$\geq$" $g(n)$
- **θ** notation: asymptotic "equality":
  - $f(n)= \theta (g(n))$ implies: $f(n)$ "$=$" $g(n)$

ASYMPTOTIC NOTATIONS

O-notation

$$O(g(n)) = \{f(n) : \text{ there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le f(n) \le cg(n) \text{ for all } n \ge n_0\} .$$

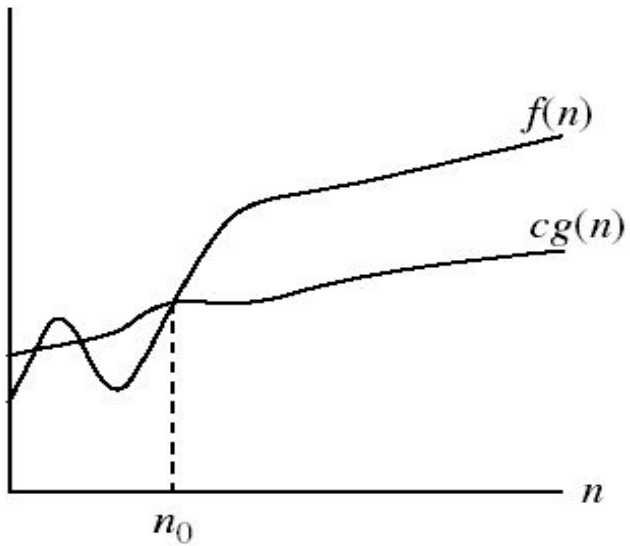


$g(n)$ is an ***asymptotic upper bound*** for $f(n)$.

ASYMPTOTIC NOTATIONS (CONT.)

Ω - notation

$$\Omega(g(n)) = \{f(n) : \text{ there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le cg(n) \le f(n) \text{ for all } n \ge n_0\} .$$



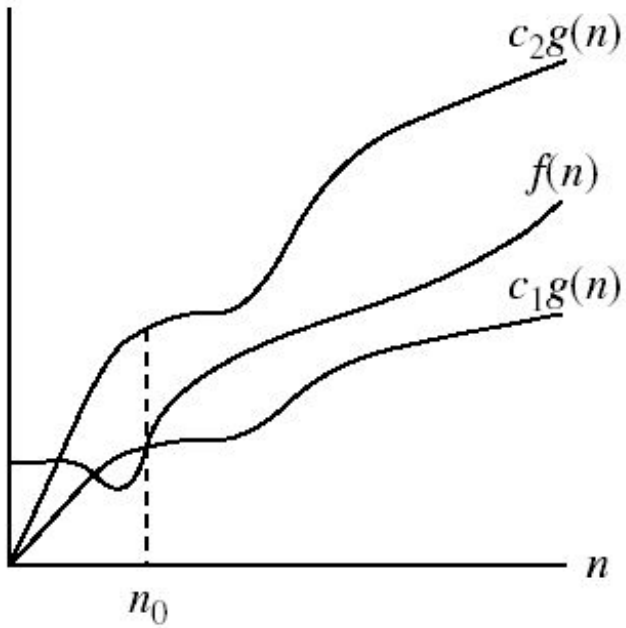
Ω(g(n)) is the set of functions with larger or same order of growth as g(n)

$g(n)$ is an ***asymptotic lower bound*** for $f(n)$.

ASYMPTOTIC NOTATIONS (CONT.)

θ –*notation*

$$\Theta(g(n)) = \{f(n) : \text{ there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \le c_1 g(n) \le f(n) \le c_2 g(n) \text{ for all } n \ge n_0\} .$$



θ(g(n)) is the set of functions with the same order of growth as g(n)

$g(n)$ is an ***asymptotically tight bound*** for $f(n)$.

Exercise

- Show that $\frac{1}{2}n^2+3n = \Theta(n^2)$

2. Show that $\frac{1}{2}n(n-1) = \Theta(n^2)$
3. Show that $10n^2+4n+3 \neq O(n)$
4. Show that $(n \log n - 2n + 13) = \Omega(n \log n)$
5. Show that $\frac{1}{2}n^2 - 3n = \Theta(n^2)$

Methodology of Analysis of Algorithm

2. **Recurrence Relation**

A recurrence is an equation or inequality that describes a function in terms of its values on smaller inputs. For example,

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(\frac{n}{2}) + \Theta(n) & \text{if } n>1 \end{cases}$$

There are three methods for solving a recurrence relation.

1. Substitution method
2. Iteration Method
3. Recursion Tree Method
4. Master Method

Note: When we solve recurrence, we often omit floors and ceiling boundary conditions.

SUBSTITUTION METHOD

The substitution method for solving recurrences comprises two steps:

1. Guess the form of the solution.
2. Use mathematical induction to find the constants and show that the solution works.

Substitution method can be used to establish either upper or lower bounds on a recurrence. As an example, let us determine an upper bound on the recurrence

$$T(n) = 2T(\lfloor \frac{n}{2} \rfloor + n$$

Here we guess that the solution is $T(n) = \bar{O}(n \lg n)$. The substitution method requires us to prove that $T(n) \leq cn \lg n$ for an appropriate choice of the constant $c > 0$.

HOW TO MAKE A GOOD CHOICE

There is no general way to guess the correct solutions to recurrences. Guessing a solution takes experience and, occasionally, creativity. Fortunately, though, you can use some heuristics to help you become a good guesser. If a recurrence is similar to one you have seen before, then guessing a similar solution is reasonable. Consider the recurrence

$$T(n) = 2T(\lfloor \frac{n}{2} \rfloor + 17) + n$$

Which is similar to the previous relation. So, we make the guess that $T(n) = \bar{O}(n \lg n)$, which you can verify as correct by using the substitution method.

PRACTICE SET-2

- Show that the solution of $T(n) = 2T(\frac{n}{2})+n$ is $O(n\log n)$
2. Show that the solution of $T(n) = T(n-1)+n$ is $O(n^2)$.
3. Show that the solution of $T(n) = 2T(\lfloor n/2 \rfloor+17) +n$ is $O(n\lg n)$.
4. Consider the recurrence $T(n) = 2T\lfloor n/2 \rfloor+1$, show that it is asymptotically bound by $O(\log n)$.

Iteration Method

- Expand the recurrence and express it as a summation of terms dependent only on 'n' and initial condition.

1. $T(n) = 3T(\frac{n}{4})+n$
2. $T(n) = T(n-1)+1$ and $T(1) = \Theta(1)$.
3. $T(n) = T(\frac{n}{3}) + n^{4/3}$. Solve the recurrence by iteration method.
4. $T(n) = T(n-1)+\frac{1}{n}$

Recursion Tree

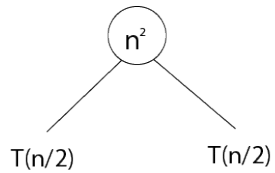
- Recursion tree is the pictorial representation of an iteration method, which is in the form of tree.

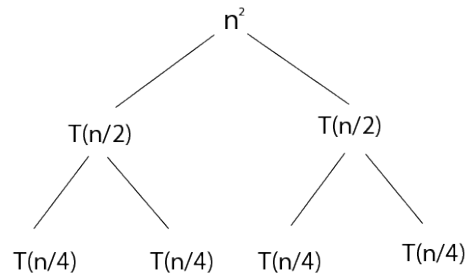
1. $T(n) = 2T(\frac{n}{2}) + n^2$
2. $T(n) = 4T(\lfloor\frac{n}{2}\rfloor + n)$
3. $T(n) = 3T(\frac{n}{4}) + n$

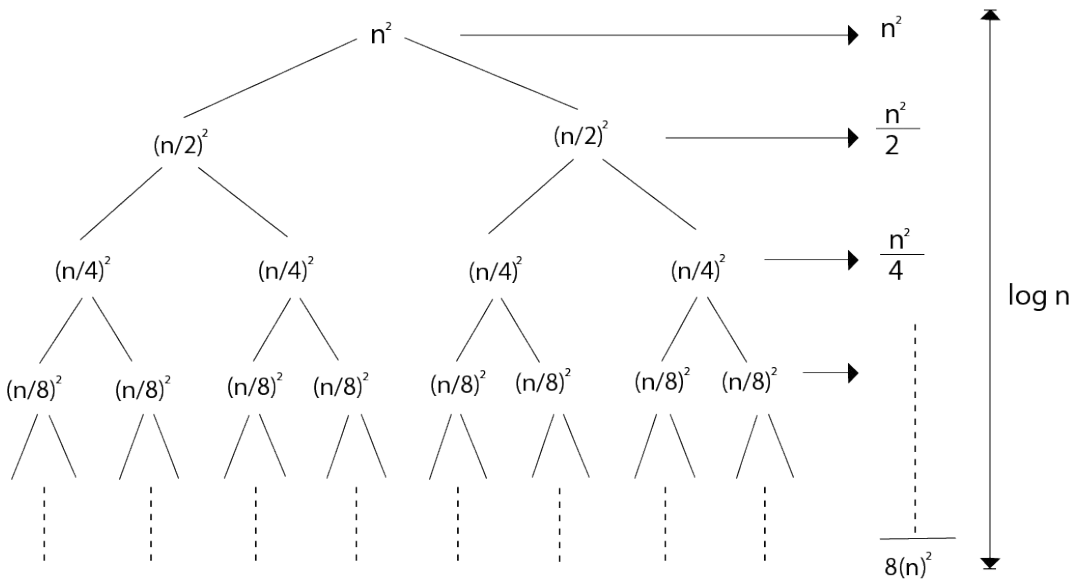
Obtain the asymptotic bound of the above recurrence relation using recursion tree method.

1. $T(n) = 2T(\frac{n}{2}) + n^2$

Solution: The Recursion tree for the above recurrence is:







$$T(n) = 2T(\frac{n}{2}) + n^2$$

- $T(n) = n^2 + \frac{n^2}{2} + \frac{n^4}{2} + \cdots logn\ times.$

$$\leq n^2 \sum_{i=0}^{\infty} (\frac{1}{2^i})$$

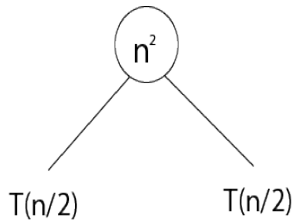
$$\leq n^2 (\frac{1}{1-1/2})$$

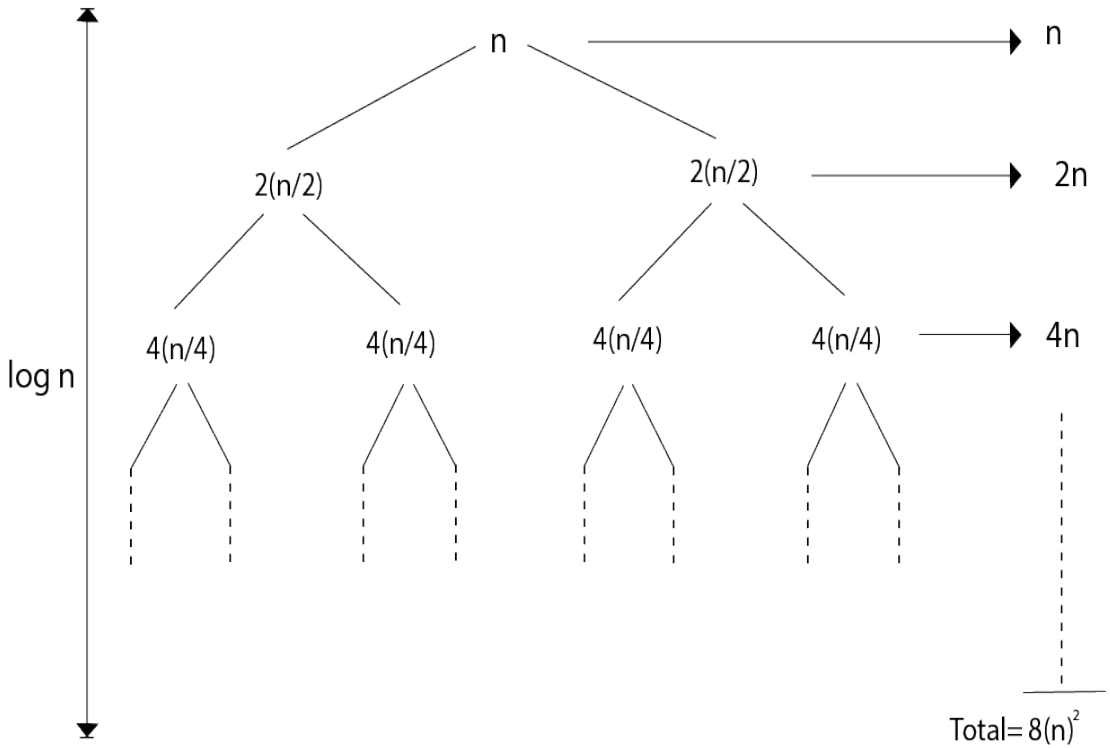
$$\leq 2n^2$$

$T(n) = O(n^2)$

2. $T(n) = 4T(\lfloor \frac{n}{2} \rfloor + n)$

The recursion trees for the above recurrence





$$T(n) = 4T(\lfloor \frac{n}{2} \rfloor + n)$$

- We have n+2n+4n……$\log_2 n$ times.

$$= n(1+2+4\ldots\ldots\ldots\log_2 n \text{ times.}$$

$$= n\left(\frac{2^{\log_2 n-1}}{2-1}\right)$$

$$= n(n-1) = n^2 - n$$

$$= \Theta(n^2)$$

Master's Method

- The master method is used for solving the following type of recurrence. It is the best method to quickly find the algorithm's time complexity from its recurrence relation.

$$T(n) = aT(\frac{n}{b})+f(n) \text{ with } a\geq 1 \text{ and } b>1.$$

The master's method depends on the following theorm.

Let T (n) is defined on non-negative integers by the recurrence.

$T(n) = aT(\frac{n}{b})+f(n)$ where $a\geq 1$ and $b>1$ be constant and f(n) be a function and $\frac{n}{b}$ can be interpreted as $\lfloor\frac{n}{b}\rfloor$ or $\lceil\frac{n}{b}\rceil$.

Master's Method

- Then can be bound asymptotically as follows:

1. If f(n) = O($n^{\log_b a-\epsilon}$) for some constant $\epsilon > 0, then\ T(n) = \theta(n^{\log_b a})$.
2. If f(n) = $\theta(n^{\log_b a})$, $then\ T(n) = \theta(n^{\log_b a}$ lgn).
3. If f(n) = O($n^{\log_b a+\epsilon}$) for some constant $\epsilon > 0,\ \ and\ if\ f\left(\frac{n}{b}\right) \leq cf(n) for\ some\ c < 1\ and\ all\ sufficiently\ large\ n,$ $then\ T(n) = \theta(f(n))$.

Practice Set-3

- $T(n) = 9T(\frac{n}{3}) + n$

  Compare the given recurrence with the $T(n) = aT(\frac{n}{b}) + f(n)$

  we get $a = 9$, $b = 3$, $f(n) = n$

Now, $n^{\log_b a} = n^{\log_3 9} = n^2$

Now compare f(n) & $n^{\log_b a}$.

$$n = O(n^{2-\varepsilon}) \text{ for } \varepsilon > 0$$

Hence it lies in case 1, and the solution is

$$T(n) = \Theta(n^2).$$

2. $T(n) = T(\frac{2n}{3}) + 1$

   Compare the given recurrence with the $T(n) = aT(\frac{n}{b}) + f(n)$

- we get a = 1, b = 3/2, f(n) = 1

Now, $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$

Now compare f(n) & $n^{\log_b a}$.

$$1 = \Theta(1)$$

Hence it lies in case 2, and the solution is

$$T(n) = \Theta(\log n).$$

- $T(n) = 3T(\frac{n}{4}) + n$
- $T(n) = 16T(\frac{n}{4}) + n^3$
- $T(n) = 7T(\frac{n}{3}) + n^2$
- $T(n) = T(n-1) + n$
- $T(n) = T(n^{1/2}) + 1$

SORTING AND ORDER STATICS

Heapsort : Heapsort introduces another algorithm design technique using a data structure, call a “heap,” to manage information.

Heaps: The (binary) heap data structure is an array object that we can view as a nearly complete binary tree. Each node of the tree corresponds to an element of the array. The tree is completely filled on all levels except possibly the lowest, which is filled from the left up to a point.

An array A that represents a heap is an object with two attributes:
A:length, which (as usual) gives the number of elements in the array, and

A:heap-size, which represents how many elements in the heap are stored within array A.

heap-size[A] = length[A] = number of elements

SORTING AND ORDER STATICS :Heapsort

The root of the tree A[1] and given index I of a node , the indices of its parent, left child and right child can be computed

PARENT (i)

return ($\lfloor i \rfloor$)

LEFT (i)

return($\lceil i \rceil$)

INSERTION SORT

- Insertion sort is a simple, **in-place, stable sorting algorithm** that works by building a sorted subarray one element at a time. It is an example of the **decrease and conquer** algorithmic paradigm.
  Algorithm Description
- The algorithm conceptually divides the input array into two parts: a sorted part and an unsorted part.
- **Initialization:** The first element of the array is considered already sorted.
- **Iteration:** The algorithm iterates from the second element to the last element of the array.
- **Key Selection:** In each iteration, the current element from the unsorted part is selected as the key.
- **Comparison and Shifting:** The key is compared with the elements in the sorted part, moving backward (from right to left). If an element in the sorted part is greater than the key, it is shifted one position to the right to make space.
- **Insertion:** This shifting continues until an element smaller than or equal to the key is found, or the beginning of the array is reached. The key is then inserted into that position.
- **Completion:** The process repeats until all elements from the unsorted part have been inserted into the sorted part.

- **Key Characteristics:**
  - **In-Place:** It requires only a constant amount of additional memory space, O(1).
  - **Stable:** It preserves the relative order of elements with equal values.
  - **Adaptive:** It is efficient for data sets that are already substantially sorted.

Linear Search

Linear search, also known as sequential search, is a fundamental algorithm in the design and analysis of algorithms that **sequentially checks each element** in a list or array until the target value is found or the entire list is exhausted. It is notable for its simplicity and ability to work on unsorted data, though it is inefficient for large datasets.

- **How Linear Search Works**
- The process involves a simple, step-by-step comparison:
- **Start** at the beginning of the data structure (e.g., the first element of an array).
- **Compare** the current element with the target value.
- **If** they match, the search is successful, and the algorithm returns the position (index) of the found element.
- **If** they do not match, move to the next element and repeat steps 2 and 3.
- **If** the entire data structure is traversed without finding a match, the search ends, typically by returning a value like -1 to indicate the element is not present.

Divide & Conquer

- Divide: Divide the array A[p…r] into two subarrays A[p….q-1] and A[q+1…r] such that each element of A[p….q-1] is less than or equal to A[q] and A[q+1…r] is greater than A[q]. Compute the index q as part of this partitioning procedure.
- Conquer: Sort the two subarrays A[p….q-1] and A[q+1…r] recursive calls to quicksort.
- Combine: Because the subarrays are already sorted, no work is needed to combine them: the entire array A[p…r] is now sorted.

Quicksort

```
QUICKSORT(A, p, r)
if p < r
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q − 1)
    QUICKSORT(A, q + 1, r)
```

To sort an entire array A, the initial call is QUICKSORT(A, 1, $A.length$).

Partitioning the array

The key to the algorithm is the PARTITION procedure, which rearranges the subarray $A[p\,..\,r]$ in place.

```
PARTITION(A, p, r)
x = A[r]
i = p − 1
for j = p to r − 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```

1. Illustrate the operation of PARTITION on the array

   A =[13,19,9,5,12,8,7,4,21,2,6,11]

2. What is the running time of QUICKSORT when all elements of array A have the same value?

3. Show that the running time of QUICKSORT is ,Θ(n2) when the array A contains distinct elements and is sorted in decreasing order.

Performance of Quicksort

Depends on the array , whether it is balanced or unbalanced. which in turn depends on which elements are used for partitioning.

- **Worst-case partitioning:** The worst-case behavior for quicksort occurs when the partitioning routine produces one subproblem with (n-1) elements and one with 0 elements. Since the recursive call on an array of size 0 just returns, T.0/ D ,.1/, and the recurrence for the running time is:

$$T(n) = T(n-1)+T(0)+\Theta(n)$$
$$= T(n-1)+ \Theta(n)$$

- **Best Case Partition:** PARTITION produces two subproblems, each of
- size no more than n=2, since one is of size n/2 and one of size (n/2-1). In this case, quicksort runs much faster. The recurrence for the running time is then

$$T(n) = 2T(n/2)+\Theta(n)$$

- Solve it by using master's method and we get
- $T(n) = \Theta(n\log n)$
- **Average partitioning:** The average-case running time of quicksort is much closer to the best case. Suppose, for example, that the partitioning algorithm always produces a 9-to-1 proportional split.
- $T(n)=T(9n/10)+T(n/10)+ \Theta(n)$

Breadth-first search

- Breadth-first search: Breadth-first search is one of the simplest algorithms for searching a graph and used for many important graph algorithms.
- Prim's minimum-spanningtree algorithm and Dijkstra's single-source shortest-paths algorithm use ideas similar to those in breadth-first search.

- Given a graph G=(V,E) and a distinguished *source vertex s, breadth-first* search systematically explores the edges of G to "discover" every vertex that is reachable from s. Breadth-first search is so named because it expands the frontier between discovered and undiscovered vertices uniformly across the breadth of the frontier.
- Breadth-first search colors each vertex white, gray, or black. All vertices start out white and may later become gray and then black.

to manage the set of gray vertices.

BFS(G, s)

```
for each vertex u ∈ G.V − {s}
    u.color = WHITE
    u.d = ∞
    u.π = NIL
s.color = GRAY
s.d = 0
s.π = NIL
Q = ∅
ENQUEUE(Q, s)
while Q ≠ ∅
    u = DEQUEUE(Q)
    for each v ∈ G.Adj[u]
        if v.color == WHITE
            v.color = GRAY
            v.d = u.d + 1
            v.π = u
            ENQUEUE(Q, v)
    u.color = BLACK
```

What is Graph Traversal?

The process of visiting and exploring a graph for processing is called traversal.

Breadth First Search(BFS)

Depth First Search(DFS)